#==========
==========

START - Testing Protocol - DO NOT EDIT OR REMOVE THIS SECTION

#==========
==========

THIS SECTION CONTAINS CRITICAL TESTING INSTRUCTIONS FOR BOTH AGENTS

BOTH MAIN_AGENT AND TESTING_AGENT MUST PRESERVE THIS ENTIRE BLOCK

Communication Protocol:

If the `testing_agent` is available, main agent should delegate all testing tasks to it.

You have access to a file called `test_result.md`. This file contains the complete testing state

and history, and is the primary means of communication between main and the testing agent.

Main and testing agents must follow this exact format to maintain testing data.

The testing data must be entered in yaml format Below is the data structure:

user_problem_statement: {problem_statement}

backend:

- task: "Task name"

implemented: true

working: true # or false or "NA"

file: "file_path.py"

stuck_count: 0

priority: "high" # or "medium" or "low"

needs_retesting: false

status_history:

-working: true # or false or "NA"

-agent: "main" # or "testing" or "user"

-comment: "Detailed comment about status"

frontend:

- task: "Task name"

implemented: true

working: true # or false or "NA"

file: "file_path.js"

stuck_count: 0

priority: "high" # or "medium" or "low"

needs_retesting: false

status_history:

-working: true # or false or "NA"

-agent: "main" # or "testing" or "user"

-comment: "Detailed comment about status"

metadata:

created_by: "main_agent"

version: "1.0"

test_sequence: 0

run_ui: false

test_plan:

current_focus:

- "Task name 1"

- "Task name 2"

stuck_tasks:

- "Task name with persistent issues"

test_all: false

test_priority: "high_first" # or "sequential" or "stuck_first"

agent_communication:

-agent: "main" # or "testing" or "user"

-message: "Communication message between agents"

Protocol Guidelines for Main agent

---

1. Update Test Result File Before Testing:

- Main agent must always update the `test_result.md` file before calling the testing agent

- Add implementation details to the status_history

- Set `needs_retesting` to true for tasks that need testing

- Update the `test_plan` section to guide testing priorities

- Add a message to `agent_communication` explaining what you've done

2. Incorporate User Feedback:

- When a user provides feedback that something is or isn't working, add this information to the relevant task's status_history

- Update the working status based on user feedback

- If a user reports an issue with a task that was marked as working, increment the stuck_count

- Whenever user reports issue in the app, if we have testing agent and task_result.md file so find the appropriate task for that and append in status_history of that task to contain the user concern and problem as well

3. Track Stuck Tasks:

- Monitor which tasks have high stuck_count values or where you are fixing same issue again and again, analyze that when you read task_result.md

- For persistent issues, use websearch tool to find solutions

- Pay special attention to tasks in the stuck_tasks list

- When you fix an issue with a stuck task, don't reset the stuck_count until the testing agent confirms it's working

4. Provide Context to Testing Agent:

- When calling the testing agent, provide clear instructions about:

- Which tasks need testing (reference the test_plan)

- Any authentication details or configuration needed

- Specific test scenarios to focus on

- Any known issues or edge cases to verify

5. Call the testing agent with specific instructions referring to test_result.md

IMPORTANT: Main agent must ALWAYS update test_result.md BEFORE calling the testing agent, as it relies on this file to understand what to test next.

#==========
==========

END - Testing Protocol - DO NOT EDIT OR REMOVE THIS SECTION

#==========
==========

#==========
==========

Testing Data - Main Agent and testing sub agent both should log testing data below this section

#==========
==========